Aristo Bendict Iskandar

ACC

10/3/2026

Stack Assignment Report

-B-

Data Structure

1st Evelyn Tjandinegara
2086022510001
Informatika (Kampus Kota Makassar)
Universitas Ciputra Surabaya
Makassar, Indonesia
etjandinegara@student.ciputra.ac.id

I. Bracket Checker

1. Problem Summary

Masalah utama yang saya alami ialah sangat susah memahami soal dan mengimplemetasikannya menjadi kode. Kemudian ada logika yang sering kali membuat saya binggung mengapa berbeda antara logika yang saya pahami dan output yang keluar.

2. Stack Design

Opening brackets yang di push ke dalam stack dengan tujuan agar memastikan urutan brackets benar. Push juga digunakan untuk menyimpan bracket yang masih terbuka. Lalu digunakan pop ketika menemukan closing brackets, sehingga program akan mengeluarkan bracket yang paling atas. Sehingga dapat dikatakan pengunaan pop dalam keadaan ini agar menandakan bracket telah menemukan pasangannya.

Dalam keadaan ini, stack selalu menyimpan opening brackets kemudian memasangkan closing brackets ke bracket yang belum memiliki pasangan.

3. Algorithm Explanation

Pertama kita buat stack kosong untuk simpan bracket yang belum memiliki pasangan. Kedua kita loop untuk baca setiap karakter. Ketiga kita berikan contoh seperti '(' , '{' , '[' kemudian jika bracket sama maka bracket akan di push ke stack kosong yang telah kita buat sebelumnya. Kemudian cek apakah bracketnya ini '(' , '{' , '[' dan jika kalau bukan, apakah bracket ini ')' , '}' , ']' . Jika stack kosong maka boolean akan return false dan mengeluarkan output "Not Balanced".

Lalu dilanjut di baris berikut, kita pop bracket yang terakhir masuk. Kemudian terjadi loop untuk mengecek pasangan. Program akan menyimpan hasil cek ke dalam 'result' kemudian jika bracket seimbang maka akan mengeluarkan output "Balanced" sedangkan jika bracket tidak seimbang, tidak memiliki pasangan ataupun kosong maka akan mengeluarkan output "Not Balanced".

4. Correctness Argument

Algortima ini bekerja berdasarkan prinsip LIFO (Last In First Out) sehingga bracket yang terakhir dimasukkan menjadi bracket yang pertama kali keluar.

Twist Requirement dalam keadaan ini memiliki arti sebagai kode yang ada harus bisa menangani berbagai bracket yang berbeda-beda bentuk secara sekaligus. Dan dalam keadaan ini kode memenuhi twist requirement karena melakukan pengecekan pasangan secara spesifik.

5. Time and Space Complexity

Time Complexity nya adalah O(n) karena loop berjalan n kali dan beberapa operasi seperti 'push()', 'pop()' , dan 'isEmpty()' bernilai konstan. Kemudian Space Complexity nya adalah O(n) karena bisa menyimpan karakter yang seukuran dengan panjang bracketnya.

6. Test Evidence

One sample test;

Input : Masukkan bracket: {[()]}
Output : Balanced

Second sample test;

Input : Masukkan bracket: {[([)]]}
Output : Not Balanced

II. Reverse Polish Notation Evaluation

1. Problem Summary

Masalah utama yang saya alami ialah sangat susah memahami soal dan mengimplemetasikannya menjadi kode. Kemudian ada logika yang sering kali membuat saya binggung mengapa berbeda antara logika yang saya pahami dan output yang keluar.

2. Stack Design

Angka dan yang bukan operator yang akan di push ke dalam stack dengan tujuan menyimpan angka untuk di operasikan ketika bertemu dengan operator. Push juga digunakan untuk menyimpan hasil dari operasi. Lalu digunakan pop ketika menemukan operator, sehingga program akan mengeluarkan dua angka yang paling atas untuk dioperasikan sesuai dengan operator yang ditemukan. Sehingga dapat dikatakan pengunaan pop dalam keadaan ini agar menandakan bracket telah menemukan pasangannya.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Dipindai dengan CamScanner

Dalam keadaan ini, stack selalu menyimpan angka kemudian menyimpan hasil perhitungan yang belum selesai.

3. Algorithm Explanation

Pertama kita buat stack kosong untuk simpan angka yang belum diproses. Kedua baca satu per satu, jika bukan operator maka angka akan dipush ke dalam stack. Kemudian jika yang muncul operator maka dua angka yang berada di dalam stack akan di pop kemudian di operasikan. Ketiga lakukan operasi matematika sesuai dengan operator yang ditemui, seperti '+' maka lakukan penjumlahan, '-' maka lakukan pengurangan, '*' maka lakukan perkalian dan '/' maka lakukan pembagian.

Ketika hasil dari dua angka tersebut telah dioperasikan maka hasil akan di push Kembali ke dalam stack. Kemudian push ke dalam stack angka berikutnya, lalu untuk operator berikutnya akan mengoperasikan dua angka, yaitu hasil sebelumnya dan angka yang baru saja ditemukan. Dan yang terakhir kita akan pop hasil akhir yang berada di dalam stack.

4. Correctness Argument

Algortima ini bekerja berdasarkan prinsip LIFO (Last In First Out) sehingga angka yang terakhir dimasukkan menjadi angka yang pertama kali keluar ketika di pop.

Twist Requirement dalam keadaan ini memiliki arti sebagai kode yang ada bisa menangani khususnya ketika penggunaan operator '/'. Dan dalam keadaan ini kode memenuhi twist requirement karena ketika melakukan pembagian yang menyisakan sisa seperti angka desimal maka bagian desimal akan dihapus.

5. Time and Space Complexity

Time Complexity nya adalah O(n) karena pengecekan angka dan operator hanya dilakukan sekali dan Space Complexity nya adalah O(n) karena stack menyimpan menyimpan hasil sementara ketika masih berjalan.

6. Test Evidence

One sample test;

Input : Masukkan RPN expression dengan spasi:4 13 5 / +
Output : Hasil akhir: 6

Second sample test;

Input : Masukkan RPN expression dengan spasi:2 1 + 3 *
Output : Hasil akhir: 9

III. Chrono Stack Engine (Stack Instruction Reconstruction)

1. Problem Summary

Masalah utama yang saya alami ialah sangat susah memahami soal dan mengimplemetasikannya menjadi kode. Kemudian ada logika yang sering kali membuat saya binggung mengapa berbeda antara logika yang saya pahami dan output yang keluar.

2. Stack Design

Ada dua jenis stack yang digunakan dalam kode ini, yaitu 'Stack<Integer> timeline' dan didalam stack ini yang akan di push ialah angka yang di input oleh user. Kemudian yang akan di pop ialah inputan user sesuai dengan prinsip LIFO (Last In First Out).

Lalu berikutnya ada 'Stack<String>' dan didalam stack ini yang di push ialah instruksi sementara. Kemudian yang akan di pop ialah instruksi dengan tetap menggunakan prinsip LIFO (Last In First Out).

Dengan ini, pengunaan stack di program membantu untuk menjaga urutan angka atau data dari user tetap sesuai dan urutan instruksi dapat terurut dengan benar. Sehingga dapat dikatakan hasil akhir instruksi sesuai dengan input yang diberikan oleh user.

3. Algorithm Explanation

Pertama program membaca jumlah angka dan nilai inputnya. Kedua setiap inputan dimasukkan ke dalam stack sehingga angka yang terakhir masuk yang akan dikerjakan terlebih dahulu. Ketiga program akan memproses angka dengan menggunakan 'pop()'. Kemudian 'pop()" akan mengambil angka yang paling terakhir masuk sehingga program akan memproses dari belakang. Hal ini juga terjadi karena pada aturan temporal stack, setiap operasi '+' menyebabkan distorsi waktu. Oleh karena itu, angka diproses dari belakang agar nilai akhirnya tetap benar. Lalu tambah target dengan jumlah distorsi yang terjadi. Selanjutnya kita buat barisan instruksi untuk membangun angka yang telah ditambahkan.

Kemudian ada dua logika, jika angka genap maka duplikatkan angka tersebut dan jika angka ganjil maka tambah kan dengan angka 1. Kemudian instruksi di simpan di stack sementara sehingga nantinya akan keluar sesuai dengan urutan yang dari depan atau dapat dikatakan dengan stack sementara maka program nantinya dapat membalik-kan urutan instruksi. Berikutnya kita hitung perubahan waktunya, lalu ditambahkan di 'akumulasiDistorsi'. Setelah itu, gabungkan semua instruksi sehingga dapat sesuai dengan urutan input. Kemudian setelah semua selesai diproses, program akan mengeluarkan output.

4. Correctness Argument

Algortima ini bekerja berdasarkan prinsip LIFO (Last In First Out) sehingga sehingga total distorsinya jauh lebih sederhana dan tetap terkendali.

Twist Requirement dalam keadaan ini memiliki arti sebagai kode yang ada akan bekerja untuk terus menambah nilai-nilai yang muncul. Dan dalam keadaan ini kode memenuhi twist requirement karena setiap karakter

Dipindai dengan CamScanner

operasional yang dihasilkan akan terus menambah nilai pada urutan berikutnya.

5. Time and Space Complexity

Time Complexity nya adalah O(n log n) karena membaca inputan memakan kompleksitas waktu sebanyak O(n) kemudian loop untuk setiap angka memakan kompleksitas waktu sebanyak O(log n) sehingga keseluruhan kompleksitas waktunya menjadi O(n log n) dan Space Complexity nya adalah O(n) karena stack seringkali hanya digunakan sebagai penyimpan sementara.

6. Test Evidence

One sample test;

Input : 2
3 1
Output : 1d+1+1

Second sample test;

Input : 5
1 5 6 3 2
Output :
1d+1+d+d+1d+1+d+d+1d+d+d+1+1d+d+1d+

IV. Cheat Buster Security System (Stack-Based Ordering)

1. Problem Summary

Masalah utama yang saya alami ialah sangat susah memahami soal dan mengimplemetasikannya menjadi kode. Kemudian ada logika yang sering kali membuat saya binggung mengapa berbeda antara logika yang saya pahami dan output yang keluar.

2. Stack Design

Angka yang di input oleh user akan di push ke dalam stack utama. Dan menggunakan pop saat proses sorting dengan mengikuti prinsip angka yang terakhir masuk yang pertama di keluarkan. Lalu dalam kode ini ada juga penggunaan push yang lain, dengan tugas memasukkan kembali angka ke dalam stack sementara.

Sehingga penggunaan, stack utama berguna untuk menyimpan semua angka yang dimasukkan oleh user atau dapat dikatakan angka yang masih belum diproses dan stack sementara untuk menampung angka yang telah terurut.

3. Algorithm Explanation

Pertama user memasuk-kan angka. Kedua push setiap angka kedalam stack. Ketiga bandingkan angka yang di 'pop ' dari stack utama dengan angka teratas di stack sementara dengan 'peek' lalu jika angka pada stack sementara lebih besar, maka letakkan angka di stack utama. Proses ini terus dilakukan hingga semua angka diproses. Hal ini membuat angka lebih terurut. Kemudian setelah semua selesai pindahkan elemen dari stack sementara ke 'ArrayList' dan kemudian dilakukan 'reverse' agar output dapat terbaca dari kecil ke besar.

Program juga menangani beberapa kasus seperti ketika stack kosong, program tidak akan melakukan loop.

4. Correctness Argument

Algortima ini bekerja berdasarkan prinsip LIFO (Last In First Out). Algortitma ini sama seperti melakukan sort tapi menggunakan insertion stack. Jadi setiap angka yang berada di stack utama akan diambil kemudian dibandingkan dengan yang berada di elemen sementara lalu diposisikan sehingga hasil akhirnya terurut dari kecil ke besar.

Twist Requirement dalam keadaan ini memiliki arti sebagai kode yang bisa melakukan sorting hanya dengan menggunakan stack dan dibantu dengan push serta pop manual. Dan dalam keadaan ini kode memenuhi twist requirement karena seluruh proses pengurutan dilakukan menggunakan stack secara manual tanpa metode sorting biasa.

5. Time and Space Complexity

Time Complexity nya adalah $O(n^2)$ karena setiap elemen perlu dibandingkan dan banyak dilakukan pemindahan lalu Space Complexity nya adalah O(n) karena stack yang digunakan akan tetap sesuai dengan seberapa banyak input an yang masuk.

6. Test Evidence

One sample test;

Input : Masukkan angka dengan spasi: 5 3 7
Output : 3 5 7

Second sample test;

Input Masukkan angka dengan spasi: 42 9 17 63 28 5 74
Output : 5 9 17 28 42 63 74

V. Operation Chastise (Stack-Based Counting)

1. Problem Summary

Masalah utama yang saya alami ialah sangat susah memahami soal dan mengimplemetasikannya menjadi kode. Kemudian ada logika yang sering kali membuat saya binggung mengapa berbeda antara logika yang saya pahami dan output yang keluar.

2. Stack Design

Stack digunakan untuk menyimpan setiap nilai. Hal ini dikarenakan nilai dihitung dari nilai terbesar menuju nilai terkecil dengan membagi dua nilai sebelumnya. Push dalam keadaan ini digunakan untuk memasukkan semua nilai hasil perhitungan ke dalam stack. Kemudian pop digunakan untuk mengeluarkan nilai dari stack ketika ingin menampilkan outputnya.

Dipindai dengan CamScanner

Stack pada keadaan ini berguna untuk menyimpan nilai *dari* yang terbesar hingga terkecil sehingga ketika di pop *dari* stack, urutan yang akan tampil mulai dari terkecil *hingga* terbesar.

3. Algorithm Explanation

Pertama program membuat stack kosong untuk menyimpan nilai momentum dari setiap pantulan bom. Kedua program akan membaca input yang dimasukkan user. Ketiga program akan melakukan perulangan ketika nilai momentum masih lebih besar atau sama dengan satu.

Kemudian nilai momentum akan dimasukkan ke dalam stack menggunakan push. Setelah itu, nilai momentum akan dibagi dua menggunakan operasi pembagi. Proses ini dilakukan hingga nilai momentum lebih kecil dari satu. Dengan itu, setiap momentum telah disimpan di dalam stack.Kemudian program akan mengeluarkan output yang tersimpan di dalam stack menggunakan pop. Nilai yang keluar akan terurut dari kecil hingga ke paling besar.

4. Correctness Argument

Algortima ini bekerja berdasarkan prinsip LIFO (Last In First Out) sehingga nilai momentum yang terakhir masuk ke dalam stack ialah nilai momentum yang paling kecil karena momentum selalu dibagi dua. Hal ini membuat hasil yang dicetak menjadi urutan dari kecil hingga besar.

Twist Requirement dalam keadaan ini memiliki arti sebagai kode yang ada harus menghitung momentum dari setiap pantulan dengan menggunakan pola pembagian dua hingga mencapai nilai satu.

5. Time and Space Complexity

Time Complexity nya adalah O(log n) karena setiap nilai momentum akan dibagi dua dan tergantung pada berapa kali nilai tersebut dapat dibagi dua hingga mencapai satu. Space Complexity nya adalah O(log n) karena stack menyimpan semua nilai momentum hasil pembagian dua sebelum dicetak.

6. Test Evidence

One sample test;

Input : Masukkan nilai momentum: 18
Output : 1 2 4 9 18

Second sample test;

Input Masukkan nilai momentum: 50
Output : 1 3 6 12 25 50

7. Code references

Tautan ke file GitHub yang tepathttps://github.com/etjandinegara-coder/DS_Stack_EvelynTjandinegara_2086022510001.git

[1] chatgpt
[2] google AI overview

Dipindai dengan CamScanner